

Processamento de uma lista encadeada com tarefas OpenMP

Matheus Marinho

4 de setembro de 2026

1. Objetivo

O programa cria uma lista encadeada cujos seis nós armazenam nomes de arquivos fictícios. Dentro de uma região paralela, o programa percorre a lista e usa `#pragma omp task` para criar uma tarefa para cada nó. Cada tarefa imprime o nome do arquivo e o identificador da thread que realmente a executou, obtido com `omp_get_thread_num()`.

Foram comparadas uma implementação propositalmente incorreta e sua correção. Um contador em cada nó permite verificar objetivamente se ele foi processado zero, uma ou várias vezes.

2. Lista encadeada e tarefas

Cada nó possui o nome, um contador usado pelo experimento e um ponteiro para o próximo nó:

```
typedef struct No {
    char nome[64];
    int processamentos;
    struct No *proximo;
} No;
```

Uma tarefa é uma unidade de trabalho que pode ser executada posteriormente por qualquer thread da equipe. A thread que encontra `#pragma omp task` cria a tarefa, mas não precisa ser a thread que a executará. Por isso, o número mostrado por `omp_get_thread_num()` identifica a executora, e a ordem das linhas não precisa seguir a ordem da lista.

3. Versão incorreta: todas as threads produzem tarefas

Uma região `parallel` faz todas as threads executarem o bloco. Ela não divide automaticamente o percurso da lista. Portanto, colocar diretamente o laço nessa região faz cada thread visitar todos os nós:

```
#pragma omp parallel default(none) shared(inicio)
{
    for (No *atual = inicio; atual != NULL; atual = atual->proximo) {
        #pragma omp task firstprivate(atual)
        processar_arquivo(atual, "sem single");
    }
}
```

Com quatro threads e seis nós, são criadas 24 tarefas: quatro para cada arquivo. Nenhum nó foi ignorado no erro específico testado, porém todos foram processados mais de uma vez. O contador é incrementado com `atomic`, de modo que a própria verificação não introduza outra condição de corrida.

Seria ainda pior compartilhar entre as threads um único ponteiro `atual` e fazer todas o modificarem. Isso causaria uma condição de corrida: algumas poderiam ler um valor enquanto outra já o alterou, levando a nós ignorados, repetidos ou até acesso inválido. Essa alternativa não foi executada porque um programa com comportamento indefinido não é uma base segura de teste.

4. Correção: uma produtora, várias executoras

`single` determina que somente uma thread da equipe percorra a lista e produza as tarefas. As outras threads não executam o bloco `single`, mas continuam livres para retirar tarefas da fila e processá-las:

```
#pragma omp parallel default(none) shared(inicio)
{
    #pragma omp single
    {
        for (No *atual = inicio; atual != NULL; atual = atual->proximo) {
            #pragma omp task firstprivate(atual)
            processar_arquivo(atual, "com single");
        }

        #pragma omp taskwait
    }
}
```

`firstprivate(Atual)` copia para cada tarefa o valor do ponteiro no momento da criação. Assim, o avanço do laço não muda o nó pertencente a uma tarefa já criada. Embora o OpenMP já torne esse ponteiro local `firstprivate` nesse contexto, declará-lo explicitamente documenta a intenção e evita que uma mudança de escopo introduza um erro sutil.

`taskwait` faz a thread produtora aguardar suas tarefas filhas. Neste programa, a barreira implícita no fim da região `parallel` também impediria a verificação e a liberação da lista antes do término das tarefas; o `taskwait` foi mantido para deixar explícito onde o conjunto produzido deve estar concluído.

5. Resultados e reflexão

O programa foi compilado sem avisos com:

```
gcc -O2 -Wall -Wextra -std=c99 -fopenmp lista_tarefas.c -o lista_tarefas.exe
```

Foram feitas três execuções com quatro threads. Em todas elas foi observado:

Versão	Tarefas criadas	Processamentos por nó	Resultado
sem <code>single</code>	24	4	todos os seis nós repetidos
com <code>single</code>	6	1	todos os seis nós exatamente uma vez

Na versão corrigida, todos os nós foram processados; nenhum apareceu mais de uma vez e nenhum foi ignorado. Na versão incorreta, todos apareceram quatro vezes, uma vez para cada thread que repetiu o percurso e criou sua própria tarefa.

Entre execuções, a contagem não mudou: o defeito da primeira versão cria exatamente uma tarefa por par *thread*–nó, enquanto a segunda cria exatamente uma por nó. Entretanto, mudaram a ordem das mensagens e as threads que executaram cada arquivo. Isso é esperado, pois o escalonamento das tarefas é dinâmico e não há garantia de ordem. Uma saída diferente, isoladamente, não significa erro; os contadores por nó é que comprovam a cobertura correta.

6. `single`, `nowait` e sincronização

Há uma barreira implícita ao final de `single`: em condições normais, as threads esperam ali. A cláusula `nowait` remove essa barreira, permitindo que sigam para o código posterior. Ela não faz várias threads executarem o bloco; o percurso continua pertencendo a uma única produtora.

Neste caso, `nowait` não é necessário. Mesmo se fosse colocado em `single`, a barreira implícita ao fim de `parallel` ainda aguardaria as tarefas antes de o programa resumir os contadores e liberar a memória. Se a lista precisasse ser verificada, alterada ou liberada ainda dentro da região paralela, seria necessário manter uma sincronização apropriada, por exemplo `taskwait`, `taskgroup` ou uma barreira, conforme a estrutura do programa.

`master` não é a melhor escolha para este padrão. Ele reserva o bloco para a thread 0 e não possui barreira implícita. `single` permite que qualquer thread disponível seja a produtora e expressa diretamente que o bloco deve ser executado uma única vez.

7. Conclusão

Criar tarefas não distribui automaticamente o código que as cria. A garantia de uma única tarefa por nó vem da combinação de três decisões: apenas uma thread percorre a lista com `single`; cada tarefa captura seu próprio ponteiro com `firstprivate`; e a memória permanece válida até todas as tarefas terminarem. O runtime continua livre para escolher qualquer thread executora e qualquer ordem, sem alterar a propriedade importante: cada nó é processado exatamente uma vez.

8. Código-fonte

lista_tarefas.c · 164 linhas

```
1 #include <omp.h>
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <string.h>
5
6 typedef struct No {
7     char nome[64];
8     int processamentos;
9     struct No *proximo;
10 } No;
11
12 static No *criar_lista(const char *nomes[], int quantidade) {
13     No *inicio = NULL;
14     No *fim = NULL;
15
16     for (int i = 0; i < quantidade; i++) {
17         No *novo = malloc(sizeof(*novo));
18         if (novo == NULL) {
19             while (inicio != NULL) {
20                 No *seguinte = inicio->proximo;
21                 free(inicio);
22                 inicio = seguinte;
23             }
24             return NULL;
25         }
26
27         snprintf(novo->nome, sizeof(novo->nome), "%s", nomes[i]);
28         novo->processamentos = 0;
29         novo->proximo = NULL;
30
31         if (inicio == NULL) {
32             inicio = novo;
33         } else {
34             fim->proximo = novo;
35         }
36         fim = novo;
37     }
38
39     return inicio;
40 }
41
42 static void liberar_lista(No *inicio) {
43     while (inicio != NULL) {
44         No *seguinte = inicio->proximo;
45         free(inicio);
46         inicio = seguinte;
47     }
48 }
49
50 static void zerar_contadores(No *inicio) {
51     for (No *atual = inicio; atual != NULL; atual = atual->proximo) {
52         atual->processamentos = 0;
53     }
54 }
55
56 static void processar_arquivo(No *no, const char *versao) {
57     int ocorrencia;
58     int thread = omp_get_thread_num();
59
60     /* O contador serve apenas para verificar repeticoes no experimento. */
61     #pragma omp atomic capture
62     ocorrencia = ++no->processamentos;
63
64     /* Evita que duas linhas de printf se misturem no terminal. */
65     #pragma omp critical(saida)
66     {
67         printf("[%s] %-20s | thread executora %d | ocorrencia %d\n",
68             versao, no->nome, thread, ocorrencia);
69     }
70 }
71
72 /*
73  * Versao propositalmente incorreta: toda thread executa o mesmo laço.
74  * Com T threads, cada no gera T tarefas e e processado T vezes.
75  */
76 static void executar_sem_single(No *inicio) {
77     #pragma omp parallel default(none) shared(inicio)
78     {
79         for (No *atual = inicio; atual != NULL; atual = atual->proximo) {
80             #pragma omp task firstprivate(atual)
81             processar_arquivo(atual, "sem single");
82         }
83     }
84 }
```

```

83     }
84 }
85
86 /*
87  * Versao correta: uma unica thread percorre a lista e produz as tarefas.
88  * A equipe inteira continua disponivel para executar essas tarefas.
89  */
90 static void executar_com_single(No *inicio) {
91     #pragma omp parallel default(none) shared(inicio)
92     {
93         #pragma omp single
94         {
95             for (No *atual = inicio; atual != NULL; atual = atual->proximo) {
96                 #pragma omp task firstprivate(atual)
97                 processar_arquivo(atual, "com single");
98             }
99
100             #pragma omp taskwait
101         }
102     }
103 }
104
105 static int mostrar_resumo(const No *inicio) {
106     int diferentes_de_um = 0;
107
108     puts("Resumo da rodada:");
109     for (const No *atual = inicio; atual != NULL; atual = atual->proximo) {
110         printf(" %-20s -> %d processamento(s)\n",
111             atual->nome, atual->processamentos);
112         if (atual->processamentos != 1) {
113             diferentes_de_um++;
114         }
115     }
116     return diferentes_de_um;
117 }
118
119 int main(int argc, char *argv[]) {
120     const char *nomes[] = {
121         "relatorio.pdf",
122         "dados.csv",
123         "imagem.png",
124         "notas.txt",
125         "programa.c",
126         "apresentacao.pptx"
127     };
128     const int quantidade = (int)(sizeof(nomes) / sizeof(nomes[0]));
129     int threads = (argc > 1) ? atoi(argv[1]) : 4;
130
131     if (threads < 1) {
132         fprintf(stderr, "Uso: %s [numero_de_threads >= 1]\n", argv[0]);
133         return EXIT_FAILURE;
134     }
135
136     No *lista = criar_lista(nomes, quantidade);
137     if (lista == NULL) {
138         fputs("Nao foi possivel criar a lista.\n", stderr);
139         return EXIT_FAILURE;
140     }
141
142     omp_set_dynamic(0);
143     omp_set_num_threads(threads);
144
145     printf("Lista com %d nos; equipe solicitada com %d threads.\n\n",
146         quantidade, threads);
147
148     puts("=== 1. SEM single: erro proposital ===");
149     executar_sem_single(lista);
150     int erros_sem_single = mostrar_resumo(lista);
151
152     zerar_contadores(lista);
153     puts("\n=== 2. COM single: versao correta ===");
154     executar_com_single(lista);
155     int erros_com_single = mostrar_resumo(lista);
156
157     printf("\nDiagnostico: sem single, %d de %d nos nao foram processados uma vez.\n",
158         erros_sem_single, quantidade);
159     printf("Diagnostico: com single, %d de %d nos nao foram processados uma vez.\n",
160         erros_com_single, quantidade);
161
162     liberar_lista(lista);
163     return (erros_com_single == 0) ? EXIT_SUCCESS : EXIT_FAILURE;
164 }

```